

SKILL WEEK-10

1 . To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

Aim : To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify program stability, and test different signal events.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t signal_received = 0;

/* Signal handler */
void signal_handler(int sig)
{
    signal_received = sig;

    if (sig == SIGINT)
    {
        printf("\nSIGINT received - Interrupt handled.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received - Termination request handled.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received - User-defined event handled.\n");
    }
}

int main()
{
    struct sigaction sa;

    printf("Signal Handling Program Started\n");
    printf("Process ID: %d\n", getpid());

    /* Clear structure */
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    /* Register SIGINT */
    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }
}
```

Output :

Terminal-1 :

```

jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_signal_actions.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra task_signal_actions.c -o task_signal_actions
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_signal_actions
Signal Handling Program Started
Process ID: 488

Signal handlers registered successfully.
Waiting for signal events....
Press Ctrl+C to test SIGINT.
^C
SIGINT received - Interrupt handled.
Program is still running and stable.

SIGUSR1 received - User-defined event handled.
Program is still running and stable.

SIGTERM received - Termination request handled.
Program is still running and stable.

SIGINT received - Interrupt handled.
Program is still running and stable.
Killed
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |

```

Terminal-2 :

```

jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ps -ef | grep task_signal_actions
jhanasri      488      301    0 04:56 pts/0    00:00:00 ./task_signal_actions
jhanasri      511      494    0 04:56 pts/2    00:00:00 grep --color=auto  task_signal_actions
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ kill -SIGUSR1 488
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ kill -SIGTERM 488
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ kill -SIGINT 488
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ps -p 488
  PID TTY          TIME CMD
  488 pts/0    00:00:00 task_signal_act
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ps -o pid,ppid,state,cmd -p 4525
  PID  PPID S CMD
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ kill -9 488
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ps -p 488
  PID TTY          TIME CMD
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |

```

Observation:

- The program successfully registered signal handlers for different signals.
- SIGINT was captured and handled without immediately terminating the program.
- SIGUSR1 was received as a user-defined signal and handled successfully.
- SIGTERM was also captured and processed by the registered handler.
- The program remained stable after handling the signals.

Result:

The signal-handling program successfully registered and executed signal handlers, handled interrupt and user-defined signals, and maintained stable execution during signal events.

2 . Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

Aim : This task demonstrates how a shell handles Ctrl+C (SIGINT) without terminating the shell itself, while forwarding the signal to the foreground job.

Program :

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_job = -1;

void handle_sigint(int sig)
{
    (void)sig;

    if (foreground_job > 0) {
        printf("\nShell: Forwarding SIGINT to foreground job (PID %d)\n",
            foreground_job);

        kill(foreground_job, SIGINT);
    } else {
        printf("\nShell: SIGINT received, but no foreground job is running.\n");
    }
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;

    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Shell: Process Started\n");
    printf("Shell: PID: %d\n", getpid());
    printf("Press Ctrl-C to interrupt the foreground job.\n\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }
}

```

 Help
  Write Out
  Where Is
  Cut
  Execute

GNU nano 7.2

```
pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    signal(SIGINT, SIG_DFL);

    printf("Foreground job started.\n");
    printf("Job PID: %d\n", getpid());
    printf("Job State: RUNNING\n");
    printf("Waiting for SIGINT...\n");

    while (1) {
        sleep(1);
    }
}

foreground_job = pid;

printf("Shell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Shell is waiting for the foreground job...\n\n");

waitpid(foreground_job, &status, 0);

if (WIFSIGNALED(status)) {
    printf("\nShell: Foreground job terminated by signal %d\n",
        WTERMSIG(status));
    printf("Shell: Job State = TERMINATED\n");
}
else if (WIFEXITED(status)) {
    printf("\nShell: Foreground job exited normally.\n");
    printf("Shell: Job State = COMPLETED\n");
}

foreground_job = -1;

printf("Shell Process is still running.\n");
printf("Shell Process Protected Successfully.\n");

return 0;
}
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T
^X Exit	^R Read File	^N Replace	^U Paste	^J

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_sigint_forward.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra task_sigint_forward.c -o task_sigint_for
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_sigint_forward
Shell Process Started
Shell PID: 798
Press Ctrl+C to interrupt the foreground job.

Shell: Foreground job PID = 799
Shell: Job State = RUNNING
Shell: Waiting for the foreground job...

Foreground job started.
Job PID: 799
Job State: RUNNING
Waiting for SIGINT....
^C
Shell: Forwarding SIGINT to foreground job (PID 799)

Shell: Foreground job exited normally.
Shell: Job State = COMPLETED
Shell: Process is still running.
Shell: Process Protected Successfully.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |
```

Observation:

- The shell process remained active while the foreground job was running.
- When Ctrl+C was pressed, SIGINT was captured by the shell.
- The shell forwarded SIGINT to the foreground process.
- The foreground job completed normally.
- The shell updated the job state to COMPLETED.
- The shell process remained protected and continued running.

Result:

The program successfully demonstrated SIGINT forwarding from the shell to the foreground job while protecting the shell process and correctly updating the foreground job state.

3 . Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Aim : We can do this as a simple shell job-control simulation. The shell captures **SIGTSTP** (**Ctrl+Z**), sends it to the foreground process, records the job as STOPPED, and later resumes it using **SIGCONT**.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_job = -1;
int job_state = 0;

/* Handle Ctrl+Z */
void handle_sigstsp(int sig)
{
    (void)sig;

    if (foreground_job > 0) {
        printf("\nShell: SIGTSTP received.\n");
        printf("Shell: Suspending foreground job (PID %d)\n",
            foreground_job);

        kill(foreground_job, SIGSTOP);

        job_state = 1;
        printf("Shell: Job State = STOPPED\n");
        printf("Shell: Job state preserved.\n");
    }
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;

    /* Register SIGTSTP handler */
    sa.sa_handler = handle_sigstsp;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGTSTP, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Shell Process Started\n");
    printf("Shell PID: %d\n", getpid());

    /* Create foreground job */

    ^G Help      ^O Write Out   ^W Where Is    ^K Cut
    ^X Exit      ^R Read File   ^\ Replace     ^U Paste
```

```
/* Create foreground job */
pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    /* Child process */
    signal(SIGTSTP, SIG_DFL);

    printf("Foreground job started.\n");
    printf("Job PID: %d\n", getpid());
    printf("Job State: RUNNING\n");
    printf("Job is working...\n");

    while (1) {
        printf("Foreground job is running...\n");
        sleep(2);
    }
}

/* Parent shell */
foreground_job = pid;

printf("\nShell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Press Ctrl+Z to suspend the job.\n\n");

/* Wait for job status changes */
waitpid(foreground_job, &status, WUNTRACED);

if (WIFSTOPPED(status)) {
    job_state = 1;

    printf("\nShell: Job PID %d has been suspended.\n",
           foreground_job);
    printf("Shell: Job State = STOPPED\n");

    printf("\nPress ENTER to resume the job.\n");
    getchar();

    printf("Shell: Sending SIGCONT to job PID %d\n",
           foreground_job);
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^\ **Replace**

^K Cut
^U Paste

GNU nano 7.2

```
foreground_job = pid;

printf("\nShell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Press Ctrl+Z to suspend the job.\n\n");

/* Wait for job status changes */
waitpid(foreground_job, &status, WUNTRACED);

if (WIFSTOPPED(status)) {
    job_state = 1;

    printf("\nShell: Job PID %d has been suspended.\n",
        foreground_job);
    printf("Shell: Job State = STOPPED\n");

    printf("\nPress ENTER to resume the job.\n");
    getchar();

    printf("Shell: Sending SIGCONT to job PID %d\n",
        foreground_job);

    kill(foreground_job, SIGCONT);

    job_state = 2;

    printf("Shell: Job State = RUNNING\n");
    printf("Shell: Job resumed successfully.\n");

    sleep(5);

    printf("\nShell: Terminating foreground job.\n");
    kill(foreground_job, SIGTERM);

    waitpid(foreground_job, &status, 0);

    printf("Shell: Job State = TERMINATED\n");
}

foreground_job = -1;

printf("Shell: Job removed from foreground.\n");
printf("Shell Process completed.\n");

return 0;
```

5 Help ^O Write Out ^W Where Is ^K Cut
6 Exit ^X Read File ^_ Undo ^_ Redo

Output :

```
jhanasri@LAPTOP-1RN1M8E3: ~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_sigtstp.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra task_sigtstp.c -o task_sigtstp
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_sigtstp
Shell Process Started
Shell PID: 834

Shell: Foreground job PID = 835
Shell: Job State = RUNNING
Press Ctrl+Z to suspend the job.

Foreground job started.
Job PID: 835
Job State: RUNNING
Job is working...
Foreground job is running...
Foreground job is running...
Foreground job is running...
^Z
Shell: SIGTSTP received.
Shell: Suspending foreground job (PID 835)
Shell: Job State = STOPPED
Shell: Job state preserved.

Shell: Job PID 835 has been suspended.
Shell: Job State = STOPPED

Press ENTER to resume the job.
Enter
Shell: Sending SIGCONT to job PID 835
Shell: Job State = RUNNING
Shell: Job resumed successfully.
Foreground job is running...
Foreground job is running...
Foreground job is running...

Shell: Terminating foreground job.
Shell: Job State = TERMINATED
Shell: Job removed from foreground.
Shell: Process completed.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |
```

Observation:

- The foreground job started in the RUNNING state.
- Pressing Ctrl+Z generated SIGTSTP.
- The shell captured the signal and suspended the foreground process.
- The job state was changed from RUNNING to STOPPED.
- The job information/state was preserved.
- On pressing Enter, the shell sent SIGCONT to the stopped process.
- The job resumed successfully and returned to the RUNNING state.
- The foreground job was subsequently terminated and removed from the foreground job table.

Result:

The program successfully demonstrated process suspension and resumption using **SIGTSTP** and **SIGCONT**, with correct job-state management.

4. Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

Aim : This program demonstrates basic job-control process groups. The parent acts as the shell, creates a child process group, transfers terminal control to that group, sends a signal to the group, and then restores terminal control to the shell.

Program :

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

pid_t shell_pgid;
pid_t child_pgid;

/* Handle Ctrl+C */
void handle_sigint(int sig)
{
    (void)sig;

    printf("\nShell: SIGINT received.\n");

    if (child_pgid > 0) {
        printf("Shell: Forwarding SIGINT to process group %d\n",
            child_pgid);

        kill(-child_pgid, SIGINT);
    }
}

int main()
{
    pid_t pid;
    int status;

    shell_pgid = getpgrp();

    printf("Shell Process Started\n");
    printf("Shell PID      : %d\n", getpid());
    printf("Shell Group ID: %d\n", shell_pgid);

    /* Register SIGINT handler */
    signal(SIGINT, handle_sigint);

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

```

^G Help	^O Write Out	^W Where Is	^K Cut
^X Exit	^R Read File	^_\ Replace	^U Paste

```
/* Parent */
child_pgid = pid;

/* Make sure child is in its own process group */
setpgid(pid, pid);

printf("\nShell: Child created.\n");
printf("Shell: Child PID      = %d\n", pid);
printf("Shell: Child Group ID = %d\n", getpgid(pid));

/*
 * Transfer terminal control to child process group.
 * tcsetpgrp() may fail when the program is not running
 * with a controlling terminal, so report the error.
 */
if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1) {
    perror("tcsetpgrp - transfer terminal");
} else {
    printf("Shell: Terminal control transferred to child group.\n");
}

sleep(3);

printf("\nShell: Sending SIGINT to child process group.\n");
kill(-child_pgid, SIGINT);

sleep(2);

/* Restore terminal control to shell */
if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
    perror("tcsetpgrp - restore terminal");
} else {
    printf("Shell: Terminal control restored.\n");
}

/* Terminate child */
kill(-child_pgid, SIGTERM);

waitpid(pid, &status, 0);

printf("Shell: Child process terminated.\n");
printf("Shell: Process group management completed.\n");

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

^T Exec
^J Just

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_process_groups.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra task_process_groups.c -o task_process_groups
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_process_groups

Shell Process Started
Shell PID      : 1045
Shell Group ID: 1045

Shell: Child created.
Shell: Child PID   = 1046
Shell: Child Group ID = 1046
Shell: Terminal control transferred to child group.

Child Process Started
Child PID      : 1046
Child Group ID: 1046
Child: Waiting for signals...
```

[illegible]

```
jhanasri@LAPTOP-1RN1M8E3:~$ ps -ef | grep task_process_groups
jhanasri      888          1 89 05:33 ?                00:07:39  ./task_process_groups
jhanasri     1005          1 88 05:38 ?                00:02:50  ./task_process_groups
jhanasri     1045       1016  0 05:40 pts/3          00:00:00  ./task_process_groups
jhanasri     1046       1045  4 05:40 pts/3          00:00:05  ./task_process_groups
jhanasri     1076       1057  0 05:42 pts/4          00:00:00  grep --color=auto task_process_groups
jhanasri@LAPTOP-1RN1M8E3:~$ kill -9 1046
```

Observation:

- The shell process was created with its own PID and process group ID.
- A child process was successfully created with a separate process group.
- Terminal control was transferred from the shell to the child process group.
- The child process displayed its PID and group ID and waited for signals.
- Process information was verified using `ps`.
- The child process could be terminated using its PID.
- The shell regained control after the child process was terminated.

Result:

The program successfully demonstrated process-group creation, group ID management, terminal-control transfer, signal coordination, and foreground process management.

